

WEEK-1 Exercises

Design principles & Patterns

Exercise 1: Implementing the Singleton Pattern

```
Singleton.java
import java.util.*;
class Singleton {
    private static Singleton instance;
    Singleton() {
        System.out.println("Created a Singleton Class");
    } //Private constructor
    public static Singleton getInstance(){
        if (instance==null){
            instance=new Singleton();
        }
        return instance;
    }
}
```

PORTS

▼ TERMINAL

```
● PS P:\Cognizant_exercise> javac Singleton.java Main.java
● PS P:\Cognizant_exercise> java Main
Created a Singleton Class
Both are same
❖ PS P:\Cognizant_exercise> █
```

Main Loading...

```
1 public class Main{
2     public static void main(String args[]){
3         Singleton s1=Singleton.getInstance();
4         Singleton s2=Singleton.getInstance();
5         if(s1==s2){
6             System.out.println("Both are same");
7         }
8     }
9 }
```

Exercise 2: Implementing the Factory Method Pattern

```
FactoryMethod.java
1  import java.util.*;
2  interface Animal{
3      void speak();
4  }
5  class Dog implements Animal{
6      public void speak(){
7          System.out.println("Woof");
8      }
9  }
10 class Cat implements Animal{
11     public void speak(){
12         System.out.println("Meow");
13     }
14 }
15 class FactoryMethod {
16     public static void getFactory(String type){
17         if(type=="Dog"){
18             Dog d=new Dog();
19         }
20         else if(type=="Cat"){
21             Cat c=new Cat();
22         }
23         else{
24             System.out.println("Invalid Type");
25         }
26     }
27 }
```

```
public class Main{  
    public static void main(String args[]){x:  
        Animal a=new Dog();  
        a.speak();  
        Animal b=new Cat();  
        b.speak();  
    }  
}
```

AL PORTS QUERY RESULTS

✓ **TERMINAL**

```
● PS P:\Cognizant_exercise> javac FactoryMethod.java Main.java  
● PS P:\Cognizant_exercise> java Main  
Woof  
Meow  
❖ PS P:\Cognizant_exercise> 
```